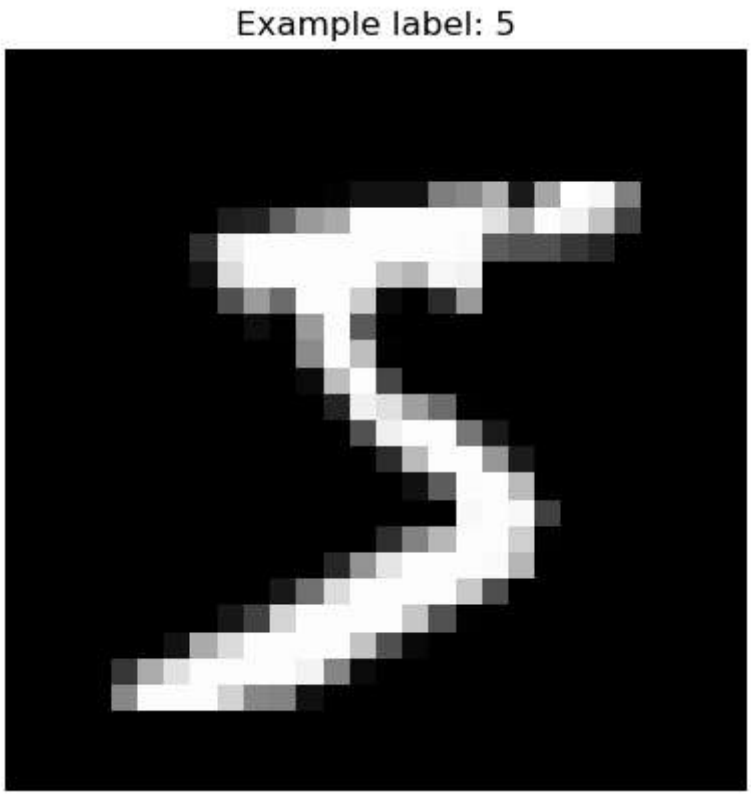
In [ ]:
```
#%pip install -q tensorflow matplotlib
```

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
print("Training images:", X_train.shape, "Testing images:", X_test.shape)
plt.imshow(X_train[0], cmap="gray")
plt.title(f"Example label: {y_train[0]}")
plt.axis("off")
plt.show()
```

```
Training images: (60000, 28, 28) Testing images: (10000, 28, 28)
```



In [2]:
```
# Normalize pixels and flatten 28x28 images into 784 values
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
model.compile(optimizer="adam",
```

```
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 64)	
dense_2 (Dense)	(None, 10)	

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

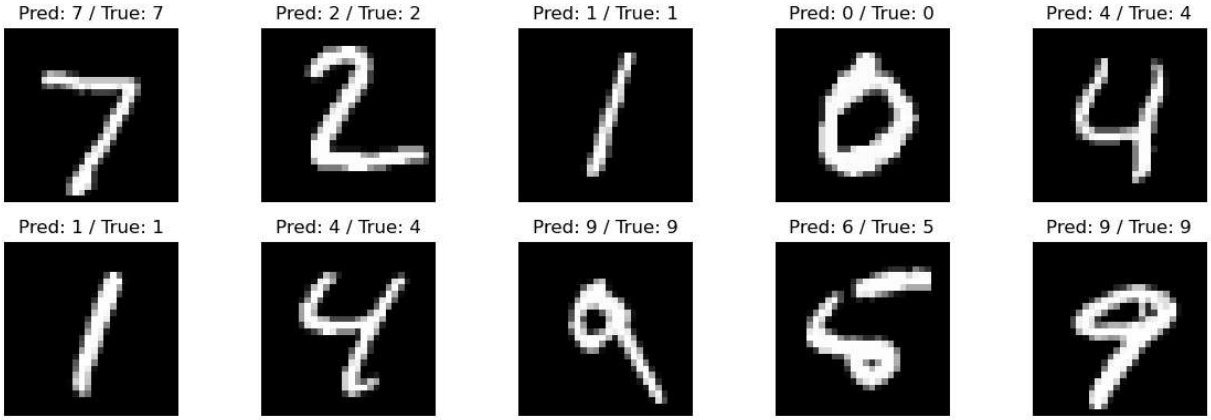
Non-trainable params: 0 (0.00 B)

In [3]:
```
history = model.fit(X_train, y_train, epochs=5, batch_size=128,
                    validation_split=0.1, verbose=1)
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")
```

```
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9006 - loss: 0.3498 - val_acc
uracy: 0.9565 - val_loss: 0.1547
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9570 - loss: 0.1474 - val_acc
uracy: 0.9680 - val_loss: 0.1095
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9696 - loss: 0.1025 - val_acc
uracy: 0.9697 - val_loss: 0.0979
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9774 - loss: 0.0761 - val_acc
uracy: 0.9755 - val_loss: 0.0883
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9819 - loss: 0.0590 - val_acc
uracy: 0.9768 - val_loss: 0.0807
Test accuracy: 0.9745
```

In [4]:
```
# Display predictions for 10 test images
probabilities = model.predict(X_test[:10], verbose=0)
predictions = np.argmax(probabilities, axis=1)
plt.figure(figsize=(12, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Pred: {predictions[i]} / True: {y_test[i]}")
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Expected result: The ANN learns to classify digits from 0 to 9 and displays predictions alongside the true labels. Accuracy depends on training settings.